SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

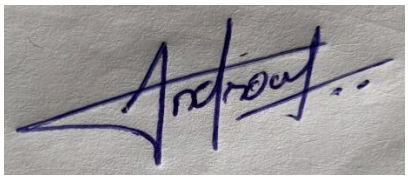
CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

I I.ANDROW EDISSON(192511136) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

1: **HTTP Response Analysis**

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.

- HTTP/1.1 200 OK: Indicates that the request was successful.
- Date: The date format is valid, but the weekday does not match the calendar date. April 1, 2026, is a Wednesday, not a Monday.
- Content-Type: text/html: Valid. It indicates that the response contains HTML content.
- Content-Length: -150: Invalid because the content length cannot be negative.
- Connection: keep-alive: Valid. It allows the connection to remain open for further requests.

2. Analyze the issue with Content-Length and explain its impact.

The Content-Length header specifies the size of the response body in bytes. Its value must be a non-negative integer.

Here, Content-Length: -150 is invalid because a negative length is not allowed.

Impact:

- The browser may not correctly determine the response body size.
- The server or client may reject or mishandle the response.
- It can cause HTTP parsing errors and communication problems.

3. Determine whether the status code matches the response correctness.

The status code 200 OK indicates that the request was successfully processed. However, the response contains an invalid Content-Length value and an incorrect weekday in the date field.

Therefore, the status code alone does not guarantee that every response header is valid. The server should correct the invalid headers before sending the response.

4. Suggest corrections for the identified errors.

The server should correct the date and provide a valid content length. For example:

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

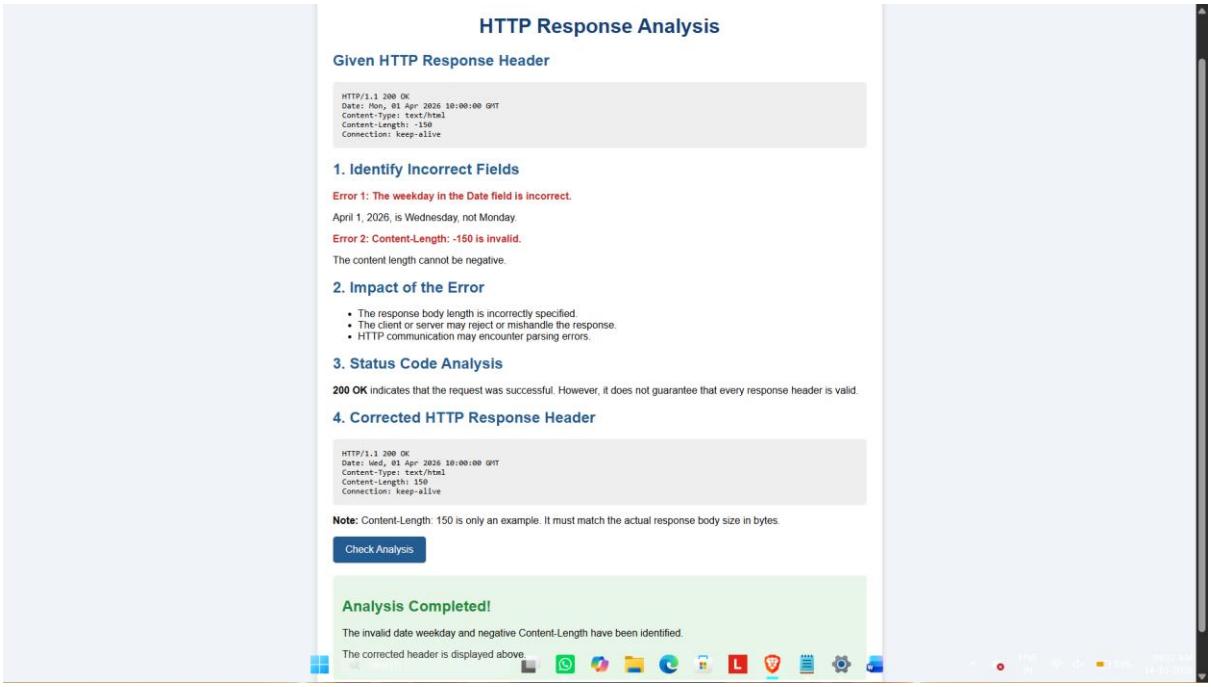
Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Here, Content-Length: 150 is an example and must match the actual response body size in bytes.

Conclusion: The HTTP response contains an invalid Content-Length value and an incorrect weekday in the date field. Correcting these fields ensures proper HTTP communication and reliable browser-server interaction.



2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.

- The (paragraph) element is missing its closing tag .
- The element is properly closed.
- The <html>, <head>, and <body> elements are properly closed.

2. Analyze how the browser will interpret this structure.

The browser attempts to correct the malformed HTML automatically. When it encounters , it will generally close the open element implicitly before closing the .

Therefore, the page may still display the text HelloWorld correctly, even though the HTML source is incomplete.

3. Determine the impact on rendering and layout.

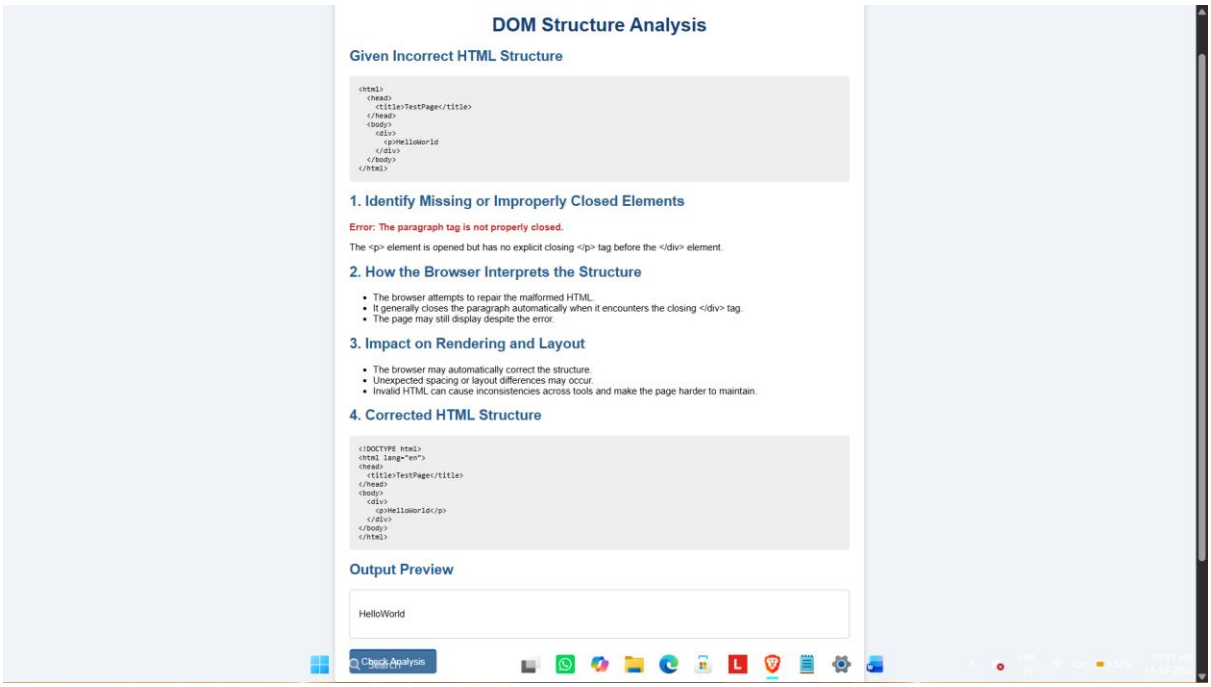
- The browser may display the content differently depending on the HTML structure.
- Missing closing tags can make the code difficult to understand and maintain.
- Incorrect HTML structure may cause unexpected layout or styling issues.
- It can create problems when JavaScript accesses or modifies the DOM.

4. Suggest corrections to make the DOM valid.

Corrected HTML Code:

```
<!DOCTYPE html>
<html>
<head>
   <title>TestPage</title>
</head>
<body>
   <div>
      <p>HelloWorld</p>
   </div>
</body>
</html>
```

Conclusion: The main error is the missing </p> tag. Adding the closing tag makes the HTML structure clear and well-formed, helping browsers render the page consistently.



3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.

GET Method:

- Sends data through the URL.
- Data is visible in the browser's address bar.
- Example: /login?user=admin&pass=1234
- It is less suitable for transmitting sensitive information.

POST Method:

- Sends data in the HTTP request body.
- Data is not displayed in the URL.
- Example: /login
- It is more suitable for submitting login information.

2. Analyze which method is more suitable for login and why.

The POST method is more suitable for login systems because usernames and passwords are sent in the request body rather than in the URL.

It reduces the risk of credentials appearing in browser history, URL logs, and shared links. However, POST alone does not provide security; HTTPS is also necessary.

3. Identify security risks in using GET for authentication.

- Passwords are visible in the URL.
- Credentials may be stored in browser history.
- URLs may be recorded in server logs.
- Sensitive information may be exposed through shared links or screenshots.
- Unauthorized persons may obtain credentials if the information is exposed.

4. Suggest best practices for secure login implementation.

- Use the POST method to submit login credentials.
- Use HTTPS to encrypt data during transmission.
- Store passwords securely using strong password hashing algorithms.
- Validate user input on the server side.
- Implement session management and secure logout.
- Use rate limiting to prevent brute-force attacks.
- Never store passwords in plain text.

Conclusion: The POST method is preferred for login systems because it sends credentials in the request body. For proper security, it must be combined with HTTPS, secure password storage, and appropriate authentication controls.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1.Identify missing closing tags.

- The <h1> tag is missing its closing tag </h1>.
- The tag is missing its closing tag .
- The <img> tag does not require a closing tag in HTML.
- The <html>, <head>, and <body> tags are properly closed.

2. Analyze issues with the <img> tag.

- The src="image.jpg" specifies the image file path.
- The image may not display if the file does not exist at the specified location.
- The alt attribute is missing.
- The alt attribute provides alternative text if the image cannot load and improves accessibility.

3. Determine how browsers will handle these errors.

- Browsers may automatically correct missing closing tags.
- The heading and paragraph may still appear correctly.
- The image will display only if the image file is available at the specified path.
- Missing alternative text reduces accessibility for users relying on screen readers.

4. Suggest a corrected version of the code.

Corrected HTML Code:

<!DOCTYPE html>

```
<html>

<head>

   <title>Sample</title>

</head>

<body>

   <h1>Welcome</h1>

   <p>This is a paragraph.</p>

   <img src="image.jpg" alt="Sample image">

</body>

</html>
```

Conclusion: The main errors are the missing closing tags for the heading and paragraph, and the missing alt attribute in the image element. Correcting these issues improves HTML structure, accessibility, and reliability.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.

Step 1: User enters a URL in the browser

- The user enters a website address in the browser.
- Example: www.google.com

Step 2: DNS lookup occurs

- DNS (Domain Name System) converts the domain name into an IP address.

- This helps the browser locate the web server.

  Step 3: HTTP request is sent

- The browser sends an HTTP request to the server to access the webpage.
- Example: GET request.

  Step 4: Server processes the request

- The server receives the request and finds the required webpage or resource.
- It processes the request and prepares a response.

  Step 5: Response is returned

- The server sends an HTTP response containing a status code, headers, and possibly webpage content.
- Example: 200 OK.

  Step 6: Browser renders the page

- The browser interprets HTML, CSS, and JavaScript.
- It displays the webpage to the user.

  2. Identify where delays can occur.

- DNS lookup: Slow DNS resolution can delay finding the server.
- Network connection: Slow internet or network congestion can delay communication.
- Server processing: A busy server may take longer to process requests.
- Data transfer: Large files and images may take longer to download.
- Browser rendering: Complex HTML, CSS, and JavaScript can delay page display.

  3. Analyze the role of DNS in the process.

  DNS stands for Domain Name System. It translates human-readable domain names into IP addresses that computers use to communicate.

  For example, when a user enters www.google.com, DNS helps the browser find the corresponding server IP address.

  Without DNS, users would generally need to remember IP addresses to access websites by address.

  4. Suggest optimizations to improve performance.

- Use DNS caching to reduce repeated DNS lookups.
- Improve internet connectivity and reduce network congestion.
- Optimize server processing and database queries.
- Compress images and other website resources.
- Use browser caching to store frequently accessed resources.
- Minimize unnecessary CSS and JavaScript.
- Use a Content Delivery Network (CDN) to serve content closer to users.

Conclusion: Browser-server communication involves URL entry, DNS lookup, HTTP request transmission, server processing, response delivery, and browser rendering. Optimizing each step improves website loading speed and user experience.

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand